

Cascadia: Resident 975B MoE Inference on Eleven AI PCs

Tate Berenbaum Matias Parij
Not Community Labs Inc. Not Community Labs Inc.

Muthaiah Venkatachalam
Intel Corporation

September 2026

Abstract

Mixture-of-experts models make nearly trillion-parameter capacity accessible with sparse per-token computation, provided that the serving system can distribute the weights and coordinate their execution. We present Cascadia’s resident execution of Inkling, a 975B-total/41B-active-parameter model, on eleven Intel Core Ultra X7 358H AI PCs, each with 64 GB of memory, Arc B390 integrated graphics and gigabit Ethernet. We contribute a custom resident MoE engine that preserves Inkling’s routing rules, constructs compressed graphs for OpenVINO’s fused iGPU primitives, and coordinates FP16 expert computation with FP32 output restoration. The engine fits six consecutive decoder layers per machine and represents dense feed-forward blocks as all-active expert slices, reducing measured dense-layer call time from approximately 8.1 to 4.5 ms. A streaming pipeline coordinates concurrent generation, while captured-state draft evaluation measures agreement with the deployed numerical path. Paired measurements at fifteen concurrency levels from 1 to 176 streams reach 60.29 aggregate decode tokens/s at 88 streams, with 46.87 tokens/s over the complete serving phases. At fifteen streams, median first-token latency is 6.05 s. Evaluation on captured fleet states separates the effects of vocabulary selection and weight quantization on draft agreement. Together, these contributions establish an execution and evaluation approach for large sparse models on distributed client systems with shared CPU–GPU memory.

1 Introduction

The sparse activation of mixture-of-experts (MoE) models changes the relationship between parameter capacity and computation. Inkling has 975B total parameters but activates 41B per token [23]. Serving such a model requires access to its expert weights even though each token selects only a subset. A collection of AI PCs offers a useful combination of distributed memory capacity, integrated matrix-compute accelerators and general-purpose CPU cores. Realizing that combination requires a serving architecture designed around shared CPU–GPU memory and sequential autoregressive dependencies.

Cascadia distributes Inkling’s text decoder across eleven Intel Panther Lake machines. Each machine holds six consecutive decoder layers and executes its own portion of the model on the local

CPU and integrated GPU. Compressed projections and fused expert operations execute on the GPU, while the CPU manages attention state, routing and stream coordination. Independent requests occupy different pipeline stages concurrently. A direct token-return connection closes the autoregressive loop, and bounded prefill windows allow prompt processing to progress through the same chain.

The research contribution is the design and evaluation of this integrated serving system at nearly trillion-parameter scale on 64 GB client devices. The work develops three aspects that are closely coupled in this setting:

1. **A custom resident MoE engine for shared-memory accelerators.** We construct compressed graphs that preserve Inkling’s routing rules while using OpenVINO’s fused expert primitives, manage numerical range across the FP16/FP32 boundary, and coordinate layer residency with distributed stream state. Dense blocks use the same fused representation through eight all-active slices, taking approximately 4.5 ms per measured call versus 8.1 ms for three compressed matrix operations.
2. **A streaming pipeline for concurrent sparse-model service.** We integrate per-stream attention and convolution state, balanced admission, eight-row prefill windows and a direct sampled-token return path. A paired concurrency curve from 1 to 176 streams characterizes the resulting service. The highest measured paired mean occurs at 88 streams, eight per pipeline group: 60.29 aggregate decode tokens/s and 46.87 whole-phase tokens/s. Fifteen streams yield 6.05 s median first-token latency.
3. **Draft evaluation aligned with deployed inference.** We capture FP32 residuals together with the fleet’s emitted token IDs and replay each draft module’s temporal state. On 36 sequences, the resulting analysis separates vocabulary selection from additional weight quantization: restricting the head to 65,536 tokens changes first-draft agreement by 2.271 percentage points, while quantization at that vocabulary changes it by a further 0.175 percentage points.

These contributions extend Cascadia’s earlier pipeline-shard and architecture work [4, 21]. The new setting combines a much larger sparse model, compressed expert residency, dense/sparse operator unification and evaluation on distributed quantized states. We position the individual mechanisms relative to consumer-cluster inference, MoE execution and speculative decoding in Section 7.

2 System architecture

2.1 Panther Lake as a shared-memory inference platform

Intel’s Core Ultra X7 358H provides sixteen cores and threads: four performance cores, eight efficient cores and four low-power efficient cores. Its Arc B390 GPU has twelve Xe cores and uses the Xe3 graphics architecture. The processor supports LPDDR5X up to 9600 MT/s and includes an NPU rated at 50 INT8 TOPS [11, 12]. Cascadia uses the CPU and integrated GPU for this deployment. CPU state processing and GPU matrix operations draw on the same local memory capacity and bandwidth.

The fleet’s aggregate capacity is distributed across independent address spaces. Model partitioning assigns weights and sequence state to each machine; network transfers carry the intermediate

Component	Deployment configuration
Fleet	Eleven Core Ultra X7 358H machines, each with nominal 64 GB memory; 704 GB nominal capacity across the fleet.
CPU and GPU	Sixteen logical CPUs and an Arc B390 integrated GPU per machine.
Usable memory	Operating-system reports of 62,757 or 62,615 MiB per machine, approximately 61.15–61.29 GiB.
Runtime	Linux, with recorded kernel 7.0.0-31-generic; OpenVINO 2026.3.1 GPU execution and Rust serving/state management.
Network	Gigabit Ethernet; eight USB NCM interfaces and three PCIe-style interfaces.
Partition	Eleven pipeline roles, each owning six consecutive decoder layers.
Serving path	Text generation; deployment default of 1024 sequence positions, evaluated with short prompts and outputs.

Table 1: The hardware and execution configuration used in the study. Vendor accelerator and memory-rate specifications describe hardware capability; serving performance is measured separately.

activations needed by the next stage. The system therefore exploits local shared memory within a machine and explicit message passing between machines.

2.2 Model structure and partitioning

Inkling’s decoder has 66 layers. Its first two feed-forward blocks are dense, with intermediate width 24,576; the remaining 64 use routed and shared experts with intermediate width 3072. Each sparse layer selects six experts from a pool of 256 and also executes two shared experts. The hidden width is 6144. Attention combines sliding and global layers, learned relative-position bias and short convolutions [24].

Pipeline role 0 owns layers 0–5, token embedding and request admission. Roles 1–9 each own six intermediate layers. Role 10 owns layers 60–65, final normalization, vocabulary projection and sampling. Forward messages contain FP32 residuals, stream-slot identifiers and positions. A width-6144 residual occupies 24,576 bytes per row before message metadata. The final role returns sampled token IDs directly to role 0, which schedules the next decoding step.

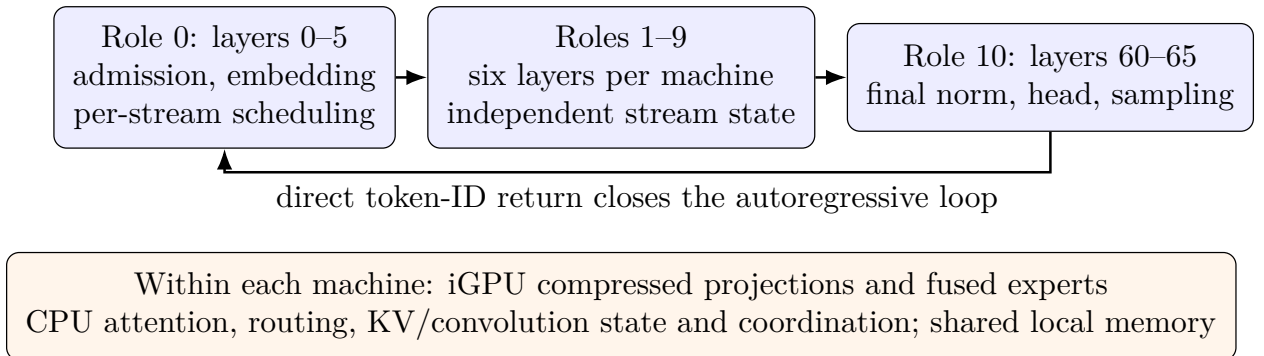


Figure 1: Cascadia’s resident streaming pipeline. Different requests can occupy different stages concurrently; consecutive ordinary tokens of one request depend on the sampled-token return.

3 A custom resident MoE engine for integrated GPUs

3.1 Model-specific graph construction and routing

Cascadia’s engine connects Inkling’s model semantics to OpenVINO’s compressed fused-MoE primitive and GPU kernel implementations [19]. Cascadia supplies the exporter, routing interface, numerical range management and resident layer runtime. OpenVINO supplies graph recognition, lowering and device execution. This division lets the engine use established accelerated operations while retaining the model’s routing and state behavior.

The exporter builds one graph per MoE layer from the existing group-32 INT4 expert bins. It arranges weights in expert-major order, preserves their packed nibbles, and represents group scales in FP16. A tiled three-matrix-operation graph expresses gate/up projections, SwiGLU, down projection and weighted reduction in the pattern recognized by OpenVINO’s MoE lowering pass [20]. This tiled algebra is a compiler representation; the fused path executes the selected expert work.

Routing remains in Rust. The gate computes Inkling’s sigmoid scores, bias-adjusted top-six selection, selected/shared normalization and model-specific scale factors. Its results enter the graph as expert IDs and routing weights. The two shared experts occupy IDs 256 and 257 and are always included, so each row supplies eight selections through the same interface. This preserves Inkling’s routing rules while allowing the GPU backend to fuse the expert computation.

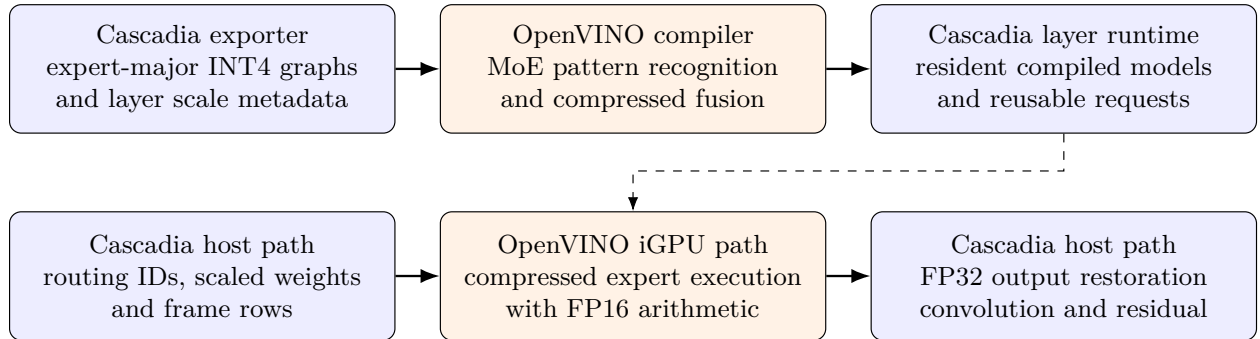


Figure 2: The custom engine and its OpenVINO execution boundary. The upper path prepares resident layer models; the lower path executes each frame. Cascadia supplies the model-specific representation and stateful runtime around OpenVINO’s compiler and GPU primitives.

3.2 Residency and the layer execution contract

Expert weights use four-bit compression; attention projections and the output head use eight-bit weights. CPU routing, attention and convolution history share local memory with GPU weights and working storage. The configuration budgets approximately 52 GiB for GPU pages per machine, with a larger allocation on the final role, and avoids a second large resident CPU expert cache in normal operation.

Cascadia’s C++ bridge materializes graph constants before compilation for the resident path. The Rust runtime retains one compiled model and reusable inference request per layer. A call supplies a frame’s rows, expert IDs and weights; the runtime selects a supported row shape, pads extra rows with zero routing weights, invokes the compiled model and restores the real row count on return. Host computation then advances the stream’s convolution and residual state. Device-call counters

and profiles make the execution placement observable. The allocation and scheduling unit is the complete layer shard, including weights, sequence state and host working memory.

3.3 Numerical range across the FP16/FP32 boundary

The engine manages two sources of magnitude separately: the expert output and its routing coefficient. For layer ℓ , the exporter can attenuate up-projection scales by $A_\ell = 2^{n_\ell}$ and record the exponent with the graph. Since the up branch enters SwiGLU multiplicatively, this scales the expert output $E_{\ell e}(x)$ by $1/A_\ell$ in real arithmetic. The recorded layer-8 configuration uses $A_8 = 16$.

For row i , with e ranging over its eight selected experts, the runtime chooses

$$F_i = 2^{\lceil \log_2 \max(1, \sum_e |w_{ie}|) \rceil}, \quad \tilde{y}_i = \sum_e \frac{w_{ie} E_{\ell e}(x_i)}{F_i A_\ell}, \quad y_i = A_\ell F_i \tilde{y}_i. \quad (1)$$

The GPU receives the normalized routing weights, whose absolute values sum to at most one. In real arithmetic, their weighted combination is bounded componentwise by the largest absolute attenuated expert value. The host restores both factors in FP32 before state updates. Layer metadata binds the export-time attenuation to the runtime compensation.

Equation 1 preserves the real-arithmetic function. Powers of two change exponents exactly when representable; FP16 conversion, subnormal scales and reduction rounding still determine finite-precision agreement. Equivalent rescaling is established numerical practice: SmoothQuant, for example, transfers activation magnitude into weights for W8A8 quantization [27]. Our implementation applies separate layer and row factors to preserve the fused FP16 expert path and the FP32 residual interface. This is an explicit engine design and implementation contribution, evaluated together with resident serving.

3.4 A common fused representation for dense and sparse blocks

The two dense feed-forward blocks can use the same fused-expert operator family as the sparse layers. Partitioning the intermediate dimension into eight slices of width 3072 gives

$$W_d[\text{SiLU}(W_g x) \odot W_u x] = \sum_{j=1}^8 W_{d,j}[\text{SiLU}(W_{g,j} x) \odot W_{u,j} x]. \quad (2)$$

Every slice is active with unit weight. Gate/up rows and down-projection columns are sliced at existing quantization-group boundaries, preserving packed weights without requantization. The dense block thus matches the backend’s expert width without introducing a learned router or dropping intermediate activations. The algebra follows established dense-to-expert decompositions [18, 31]; our implementation maps that representation onto the compressed fused operator used by the resident Inkling shard.

The two-row measurements are 8.41 ms per layer with the matrix-call path and 4.47/4.48 ms with fused slices. At the stage level, role 0’s observed computation changes from 51.9 to 43.7 ms per frame. These are operator and stage measurements; the complete serving rates reported in Section 5 characterize the distributed system separately.

Dense layer	Three matrix calls	Fused slices	Time reduction	Relative difference
Layer 0	8.15 ms	4.51 ms	44.7%	5.7×10^{-4}
Layer 1	8.11 ms	4.45 ms	45.1%	5.9×10^{-4}

Table 2: One-row load checks for the dense blocks. Relative difference compares the two FP16 execution paths. Equation 2 preserves the real-arithmetic function; the measured differences reflect finite-precision execution.

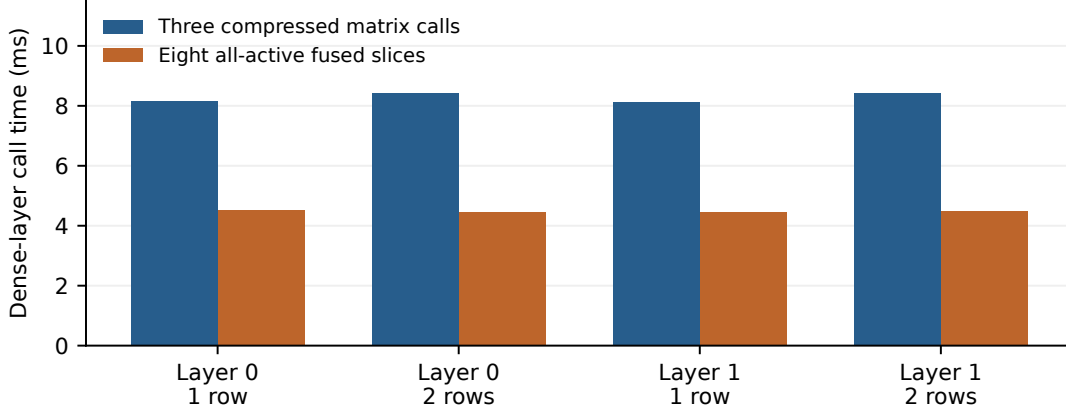


Figure 3: Dense-block execution using three compressed matrix operations or eight all-active fused slices. The operator improvement holds for the measured one- and two-row calls.

3.5 Weight traffic and the role of concurrent requests

Using the exported compressed sizes, an unshared full-model token accounts for approximately 16.76 GB of active expert weights, 8.59 GB of attention projections, 0.52 GB of dense weights and 1.24 GB of output-head weights: about 27.1 GB in total. These are logical weight bytes, including expert scale/zero-point overhead, rather than a count of physical DRAM transactions. Caching and reuse across rows determine how those bytes translate into memory traffic [26].

The layout makes two forms of parallelism explicit. Within a stage, a multi-row frame shares projection work and allows CPU attention to run across independent streams. Across stages, different frames use different memory buses concurrently. Consequently, aggregate service is a property of the distribution of requests over the pipeline as well as of the local kernels. This motivates the stream and admission design below.

4 Streaming service across the fleet

4.1 Stateful streams and direct token return

Each request occupies a stream slot with its own position, KV cache and convolution state. Frames group rows from independent slots, and each stage applies its local layers before forwarding the residuals. Role 0 admits work into the least populated group and advances the engine in short steps, allowing request submission to proceed while other frames are circulating.

The direct final-to-first connection carries sampled token IDs without requiring a reverse traversal through the intermediate workers. Receipt of an ID makes the next token of that stream schedulable. This explicit return path lets ingress interleave admission, forward progress and completion handling within the same serving loop. A full-chain readiness probe establishes downstream readiness before admission begins.

4.2 Prefill windows and parallel attention

Prompts enter as windows of at most eight rows. Consecutive windows can occupy different pipeline stages, and admission can proceed while ingress waits for token replies. The window size bounds the prefill work presented to a stage at one time. During decoding, independent rows provide parallel CPU work for attention even while the integrated GPU handles compressed projections and experts.

Chunked prefill and iteration-level scheduling have established precedents [1, 2, 30]. The contribution here is their realization within a resident sparse-model pipeline whose stages maintain both KV and convolution state, share memory between CPU and GPU, and communicate over gigabit Ethernet. Prefill, decode, row-parallel attention and direct token return are coordinated in one runtime.

4.3 Speculative decoding scenarios and measured gains

A lone active request ordinarily waits for a token to traverse all eleven stages before its next token can enter the pipeline. Cascadia overlaps this dependency by injecting proposed future input tokens behind the confirmed input, as separate frames in the same pipeline. Each returning target-model reply verifies the input of the following frame. A matching proposal retains computation already in flight; a mismatch discards that frame and its speculative successors, drains their replies and rewinds every stage’s KV and short-convolution state to the confirmed prefix. Generation resumes with the target’s token. Emitted tokens therefore come from target replies, with agreement assessed under the greedy serving configuration. This asynchronous implementation extends established speculative decoding and Cascadia’s earlier pipelined speculation [4, 13] to Inkling’s stateful layers.

Proposal scenarios. Copying and recurring phrases can obtain proposals directly from request-local history or a persisted phrase table. The hybrid proposer favors a request-local four-token suffix match or a shared three-token context seen at least twice with at least 90% next-token agreement. For other contexts, a rank-0 CPU draft model, Qwen3-0.6B in Q4_K_M format, can supply continuations. Its text is encoded with Inkling’s tokenizer before injection. Figure 4 illustrates complete agreement, partial agreement and ordinary progress when no proposal is ready. A mismatch preserves the verified prefix while restoring both forms of temporal state. When another request awaits admission, the runtime returns to its ordinary concurrent-stream path.

Why correct proposals improve latency. Let L denote full-pipeline latency, T the inter-frame interval and a the fraction of output positions with correct inputs already in flight. A first-order model gives throughput Q_{spec} and speedup S over ordinary decoding:

$$Q_{\text{spec}} \approx \frac{1}{aT + (1-a)L}, \quad S \approx \frac{L}{aT + (1-a)L}. \quad (3)$$

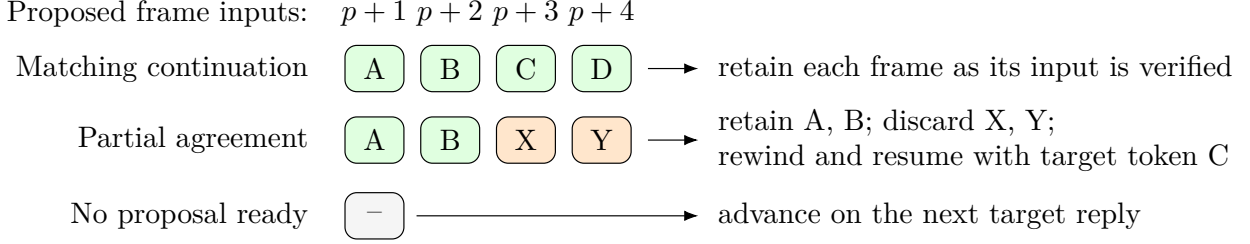


Figure 4: Speculative pipeline scenarios, assuming the target continuation is A, B, C, D. Letters denote symbolic token IDs. Frames enter successively and their inputs are verified by preceding target replies; proposed tokens are never emitted without verification.

Correct proposals can replace full round trips with shorter inter-frame intervals. Fused iGPU execution reduces the stage-computation cost, while effective proposals overlap successive positions across the eleven devices. The illustration omits additional drafting, queueing and rewind costs; a measures useful in-flight work, rather than conditional draft acceptance or offline MTP agreement.

Measured gains with the fused iGPU engine. Table 3 reports all twelve paired single-stream prompts from the finalized GPU deployment. Target experts, dense layers, attention projections and the output head use the iGPU engine; the small external drafter runs on the CPU. Each prompt produces the same complete 128-token output in both passes, with the serving binary and model fixed. The largest observed per-request rate ratio is $3.28\times$ for tips (3.55 to 11.66 tokens/s); explanation reaches 14.70 tokens/s from 5.09 ($2.89\times$), and table generation rises from 3.76 to 8.66 ($2.30\times$). Showing every pair makes the workload dependence explicit.

Prompt family	First pass	Repeated pass	Rate ratio	Gain
Explanation	5.09	14.70	$2.89\times$	+189.0%
Code	7.07	8.05	$1.14\times$	+13.9%
Arithmetic	8.88	10.06	$1.13\times$	+13.4%
Story	5.31	10.96	$2.06\times$	+106.4%
Tips	3.55	11.66	$3.28\times$	+227.9%
Table	3.76	8.66	$2.30\times$	+130.2%
Rewrite	7.36	9.58	$1.30\times$	+30.1%
Facts	6.12	10.12	$1.66\times$	+65.5%
Poem	5.13	11.69	$2.28\times$	+127.9%
Instructions	4.62	9.66	$2.09\times$	+109.0%
Translation	8.00	9.23	$1.15\times$	+15.4%
True/false	10.92	11.53	$1.06\times$	+5.6%

Table 3: Fused-iGPU target inference: first/repeated decode tokens/s for one prompt from each family, 128 output tokens per request. All twelve pairs have identical prompts and complete output text. Speculation is enabled in both passes; these are observed repeated-prompt gains with accumulated history and differing capture state.

Across the twelve serial requests, decode throughput rises from 5.68 to 10.24 tokens/s ($1.80\times$); whole-phase throughput rises from 5.19 to 8.77 ($1.69\times$). These phase rates account for the requests’ actual durations rather than averaging their individual speedups. Both passes enable speculation

and phrase learning. The first study pass already has prior history; further history accumulates before the repeat, and residual-capture writes are disabled between passes. The comparison characterizes the GPU system’s observed repeated-prompt service, with one observation per prompt per pass, rather than isolating a speculation on/off effect.

Reusable history and structured GPU workloads. An additional fused-iGPU comparison illustrates reuse of learned phrases after transferring history to the current entry role. Merging the phrase tables increases retained contexts from 24,453 to 194,299. For six previously seen 128-token prompts measured before and after the merge, code rises from 5.23 to 10.60 tokens/s ($2.03\times$) and true/false from 4.91 to 10.77 ($2.19\times$). The binary remains unchanged; repeated requests also train the table, and capture is disabled with the transfer. These observations establish reusable history within GPU serving, with those timing influences included.

The hybrid proposer also serves structured requests with related-family history: retained GPU examples reach 6.60 tokens/s for translation, 8.19 for arithmetic and 10.95 for true/false completion. A separate finalized explanation-family phase contains three 128-token single-stream requests with median 11.27 tokens/s and fastest rate 15.04 tokens/s. These are absolute request rates on the stated prompts. The GPU comparisons concern lone-stream speculation; the peak concurrent throughput in Section 5 measures a separate serving regime. The artifact retains the earlier CPU-expert on/off checks as supporting protocol evidence.

5 Evaluation of concurrent serving

5.1 Measurement protocol

The finalized survey holds the model and serving binary fixed: group-32 INT4 experts, INT8 attention/head weights and the streaming configuration described above. Thirty mixed-workload phases cover fifteen concurrency levels from 1 to 176, with two phases per level and twelve deterministic prompt families. Each request uses greedy generation and emits 128 tokens. Requests within a cohort arrive approximately 15 ms apart; at low concurrency, successive complete cohorts supply at least twelve requests per phase. The complete survey also retains three explanation-family phases, for 33 measured phases, 1,592 requests and 203,776 generated tokens. Table 4 uses the thirty mixed-workload phases.

The paired passes reuse the same prompt templates. The main sweep proceeds upward and then downward in concurrency, with the 88-stream refinement measured as an additional pair. Phrase learning remains enabled throughout. Residual-capture writes are enabled at the ascending 1–15-stream points and disabled thereafter, including the repeated pass. The pairs therefore describe the serving system with its accumulated history and recorded instrumentation state. Within high-concurrency cohorts, the finite prompt generator also produces some repeated inputs: the 88-stream phase contains 79 distinct prompts. The runtime release and binary hash, phase records and token traces accompany the artifact.

For cohort j , let f_i and l_i be the first and last token-event times of each request. The common decode interval is $(a_j, b_j]$, where $a_j = \max_i f_i$ and $b_j = \min_i l_i$. Let u_j count tokens emitted by all requests during that interval. We report

$$Q_{\text{decode}} = \frac{\sum_j u_j}{\sum_j (b_j - a_j)}, \quad Q_{\text{phase}} = \frac{\sum_i n_i}{t_{\text{end}} - t_{\text{start}}}. \quad (4)$$

Q_{decode} measures aggregate throughput while every request in a cohort is decoding. Q_{phase} includes admission, prefill, queueing and drain. Dividing Q_{decode} by concurrency gives the mean contribution per stream over these common intervals. Table and figure rates are arithmetic means of the two phase rates; TTFT quantiles pool the requests from both phases. The observed range of two phases is descriptive, rather than a confidence interval.

The client counts token-event multiplicities and checks them against each request’s final API token usage. The retained traces independently reconstruct all common intervals and whole-phase rates. TTFT refers to the first emitted model-token event, including reasoning and structural tokens. The collector checks server counters around each phase to isolate measured traffic. Software profiles supply frame-weighted stage timings, with physical device identity kept separate from pipeline role. Short serial/concurrent prefix checks and twelve known-answer questions provide functional validation; they are not a general model-quality benchmark.

5.2 Concurrency, throughput and first-token latency

Streams	Q_{decode}	Q_{phase}	Q_{decode}/C	TTFT median	TTFT p95
1	7.96	6.98	7.960	2.18	2.41
2	4.53	4.37	2.267	2.06	3.03
4	9.16	8.64	2.290	3.31	4.03
6	13.89	12.80	2.315	4.20	5.49
8	18.60	16.86	2.325	4.55	6.23
11	23.28	20.86	2.116	5.47	7.64
15	24.60	22.12	1.640	6.05	10.11
22	33.87	29.26	1.539	9.71	14.51
32	38.31	32.48	1.197	13.44	21.90
48	45.33	37.68	0.944	18.85	31.55
64	53.60	42.41	0.837	25.10	46.41
88	60.29	46.87	0.685	34.61	64.75
96	46.85	39.79	0.488	38.94	71.31
128	50.10	41.61	0.391	55.11	108.73
176	57.72	45.24	0.328	76.83	165.34

Table 4: Finalized mixed-workload concurrency curve. Every request produces 128 tokens. Rates are tokens/s; TTFT is in seconds. C is concurrent streams, rather than total requests at the low-concurrency points. Rates average two phases; TTFT quantiles pool their requests.

At 88 streams, the two phases each generate 11,264 tokens. Common-interval decode rates are 58.86 and 61.71 tokens/s; whole-phase rates are 46.03 and 47.72 tokens/s. Their paired means, 60.29 and 46.87 tokens/s, are the highest on the evaluated concurrency grid for both measures. This operating point assigns eight streams to each of eleven pipeline groups. At 176 streams the fleet sustains 57.72 aggregate decode tokens/s and 45.24 whole-phase tokens/s, while fifteen streams provide 24.60 and 22.12 tokens/s with 6.05s median TTFT.

The curve exposes a deployment-specific relationship between concurrency and the distributed execution schedule. The 88-stream alignment coincides with the highest observed throughput, and further concurrency redistributes work into larger per-group row counts. Throughput is nonmono-

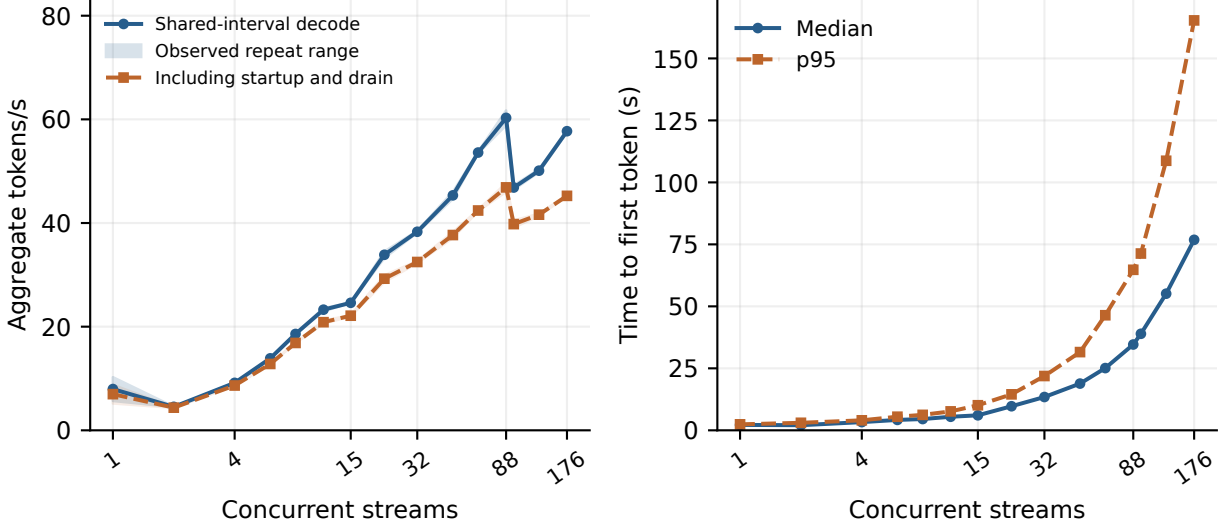


Figure 5: Paired measurements across fifteen concurrency levels. Left: common-interval decode throughput and whole-phase throughput, with shading showing the two observed phase rates. Right: pooled request TTFT median and 95th percentile. The 88-stream point is the highest paired mean on the measured grid.

tonic across that transition, while first-token latency grows as more requests enter the chain. These observations identify a useful measured operating point for this resident pipeline; they do not by themselves isolate the effects of row padding, expert routing and scheduling. The contribution is an empirical service envelope that connects the implemented architecture to explicit throughput and latency choices.

5.3 Effect of windowed admission

The reference fifteen-request burst configuration records median TTFTs of 31.52 and 31.23 s and whole-phase rates of 15.678 and 17.005 tokens/s. The windowed configuration records 6.91 s median TTFT and 22.521 whole-phase tokens/s. The median latency reduction is approximately 4.5-fold across these observed configurations.

This supporting configuration comparison uses distinct prompt tags and precedes the finalized fixed-binary survey. It characterizes the admission design; Table 4 characterizes the resulting deployment across concurrency levels. With one-second arrival spacing, the windowed configuration also records 2.48 s median TTFT for fifteen requests capped at 96 output tokens.

6 Draft evaluation on deployed states

6.1 Capturing the serving model’s actual computation

A draft head consumes internal representations whose values depend on the serving model’s quantization and execution path. We evaluate Inkling’s shipped multi-token prediction (MTP) head using states generated by the distributed fleet itself. The capture format records FP32 residuals together with positions, input token IDs and actual emitted token IDs. FP32 accommodates the

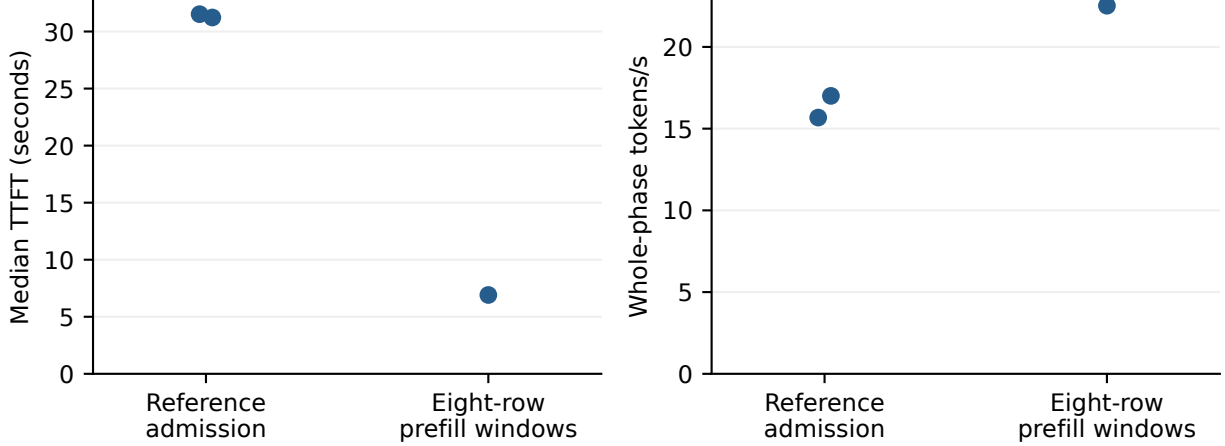


Figure 6: Fifteen-request bursts with an output cap of 128: reference admission and eight-row prefill windows. Each point is a retained phase. The comparison reports observed configurations with different prompt tags.

observed pre-final-normalization range, which includes a sampled magnitude of 4,272,968.

The corpus consists of 36 sequences, three from each of twelve task families, with 160 generated positions per sequence. The 5,760 emitted tokens provide 5,724 first-draft prediction targets. All captured residuals pass finite-value checks, prompt token IDs match the rendered inputs, and decoded full responses match the completion API output. Actual emitted IDs define the evaluation labels, preserving the numerical decisions made by the deployed model.

Each of the eight MTP modules maintains its own attention and convolution history. Replay can supply true-token context or self-fed context while preserving the module-specific temporal state. This gives a controlled basis for studying the draft’s representation, vocabulary and numerical format independently of request-serving timing.

6.2 Separating vocabulary and weight-format effects

On the same captured fleet states, the original full head yields 66.81% first-draft agreement. Restricting the original head to a prefix of 65,536 vocabulary entries yields 64.54%. Applying INT4 feed-forward and INT8 projection/head weight grids at that vocabulary yields 64.36%.

Draft calculation on captured fleet states	First-draft agreement
Original weights, full head	66.81%
Original weights, 65,536-token head	64.54%
INT4/INT8 weight grids, 65,536-token head	64.36%

Table 5: Matched-state draft evaluation over 5,724 prediction targets. The weight-grid calculation uses FP32 arithmetic offline; the table measures agreement with emitted fleet IDs, not live speculative throughput.

Vocabulary selection accounts for 2.271 percentage points of the agreement difference, and additional weight quantization at the fixed vocabulary accounts for 0.175 percentage points. This decomposition identifies vocabulary size as the larger contributor in this specific deployment-format

comparison. It provides a concrete basis for treating vocabulary footprint and draft-weight precision as separate design choices.

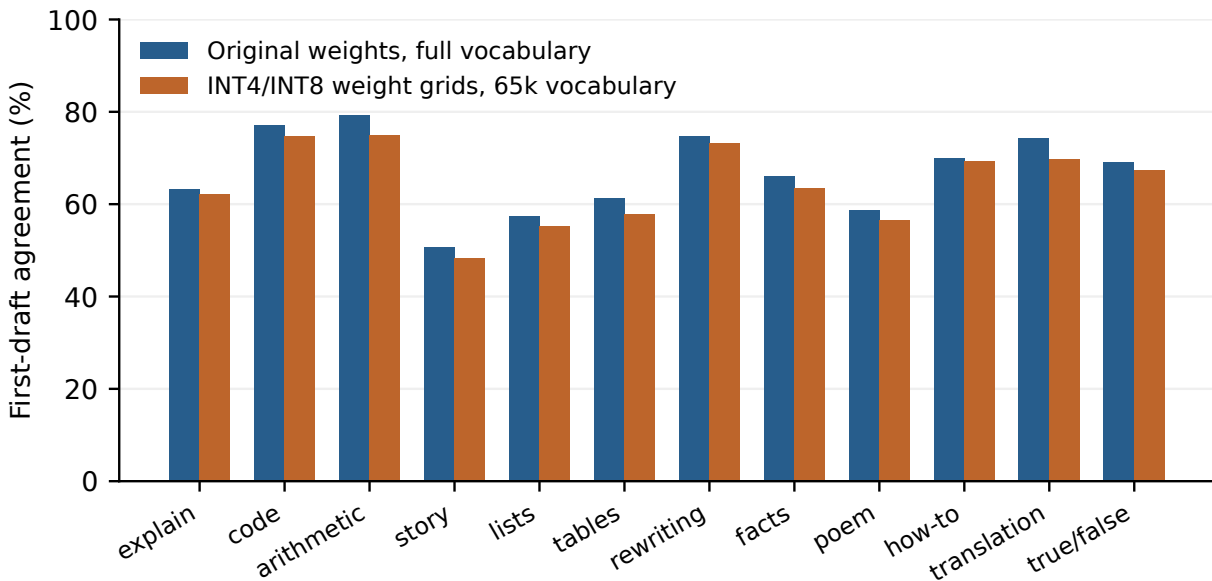


Figure 7: First-draft agreement by task family on captured fleet states. Each family contains three sequences. The two calculations share the same prediction targets and differ in draft weights and vocabulary.

The deployment-grid calculation records agreement of 0.746 for code, 0.748 for arithmetic and 0.482 for story. Across the eight-module chain, the reported expected number of accepted drafts is 2.128 with true-token context and 2.007 with self-fed context. These are descriptive offline measurements on the evaluated corpus. They complement the live phrase/model proposer results by exposing the draft head’s behavior on the distributed model’s actual states.

6.3 Contribution to draft evaluation

MTP and feature-based drafting are established approaches [9, 14]. MoE-specific speculative systems further connect acceptance, expert activation and verification cost [3, 10, 17]. Our contribution is an evaluation method and empirical decomposition tied to the numerical path of a distributed, compressed Inkling deployment: capture its residuals and emitted IDs, replay the draft’s full temporal state, and compare vocabulary and weight formats on those same labels. This connects model-level draft analysis to the exact serving configuration being studied.

7 Relationship to prior work

Client-cluster inference. Petals, TPI-LLM, prima.cpp and exo demonstrate inference across consumer or edge devices [5, 7, 15, 16]. Cascadia’s earlier work develops Intel/OpenVINO pipeline shards and the broader distributed architecture [4, 21]. The present work extends that lineage to a resident 975B sparse decoder on eleven 64 GB integrated-GPU machines, including compressed expert placement and dense/sparse operator unification. A reported Inkling deployment on eight DGX Sparks uses tensor parallelism, NVFP4, MTP and a different memory/interconnect configu-

ration [6]. The systems establish distinct deployment designs; the present evaluation uses the Intel fleet’s own workloads and numerical formats.

Sparse execution and operator representation. EdgeMoE and PowerInfer exploit sparsity and memory placement on resource-constrained hardware [22, 29]; Klotzki, MegaScale-Infer, EC2MoE and CPU/GPU hybrid serving examine expert-aware execution and distribution [8, 25, 28, 32]. Cascadia combines a resident layer partition with a CPU/iGPU split inside each shard. Its model-specific exporter and routing interface build on OpenVINO’s compressed MoE primitive and lowering passes [19, 20]; its layer/row scaling applies equivalent-rescaling principles to the FP16 expert and FP32 residual boundary [27]. MoEfication and MLPMoE establish dense-to-expert decompositions [18, 31]. The measured contribution here is using all-active slices to map Inkling’s dense blocks onto the same compressed fused operator family as its sparse blocks on Arc B390.

Streaming and draft evaluation. Orca and SARATHI/Sarathi-Serve establish iteration-level scheduling and chunked prefill [1, 2, 30]. We integrate those principles with a gigabit sparse-model pipeline, short-convolution state and a direct sampled-token return path. The draft evaluation contributes captured-state measurements of the deployed model, complementing speculative decoding, MTP and MoE-specific verification research [9, 10, 13]. The evidence supports these architectural, implementation and empirical contributions without requiring a priority claim for the underlying algebra or scheduling primitives.

8 Conclusion

Cascadia demonstrates resident inference of Inkling’s 975B MoE decoder on eleven Panther Lake AI PCs. Its custom engine preserves Inkling’s routing rules, constructs compressed graphs for OpenVINO’s fused iGPU primitives, manages numerical range across FP16 and FP32, and unifies dense and sparse blocks within resident layer shards. Stream-oriented coordination connects those shards across machines. Its paired concurrency curve reaches 60.29 aggregate decode tokens/s and 46.87 whole-phase tokens/s at 88 streams, with 6.05s median first-token latency at fifteen streams. Captured-state draft evaluation further separates vocabulary and weight-format effects using labels emitted by the deployed model. These results provide an implemented architecture and measured techniques for serving large sparse models on distributed client hardware.

Artifact. The paper source, figures, derived tables and supporting evidence are available to reviewers in the private repository <https://github.com/labscommunity/cascadia-inkling-panther-lake-paper>. The evidence is pinned to Cascadia commit 3189a189fe3428f5a6315a7eb67b13148ec314e2. `reproduction/CLAIMS.md` maps the paper’s contributions and numerical statements to source records; the analysis scripts regenerate the reported tables and figures.

References

- [1] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills, 2023. URL <https://arxiv.org/abs/2308.16369>. Accessed 2026-09-21.

- [2] Amey Agrawal et al. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve, 2024. URL <https://www.usenix.org/conference/osdi24/presentation/agrawal>. Accessed 2026-09-21.
- [3] Jehyeon Bang, Eunyeong Cho, Ranggi Hwang, Jinha Chung, and Minsoo Rhu. SpecMoE: A Fast and Efficient Mixture-of-Experts Inference via Self-Assisted Speculative Decoding, 2026. URL <https://arxiv.org/abs/2604.10152>. Accessed 2026-09-21.
- [4] Tate Berenbaum and Muthaiah Venkatachalam. Pre-Compiled Pipeline Shards for Distributed LLM Inference on Intel AI PC Fleets, 2026. URL <https://arxiv.org/abs/2608.19147>. Accessed 2026-09-21.
- [5] Alexander Borzunov, Dmitry Baranchuk, Tim Dettmers, Max Ryabinin, Younes Belkada, Artem Chumachenko, Pavel Samygin, and Colin Raffel. Petals: Collaborative Inference and Fine-tuning of Large Models, 2023. URL <https://arxiv.org/abs/2209.01188>. Accessed 2026-09-21.
- [6] emretoktas__openzeka. Inkling-NVFP4 (975B) on Eight DGX Spark Systems, 2026. URL <https://forums.developer.nvidia.com/t/inkling-nvfp4-975b-on-8x-dgx-spark/380049>. Accessed 2026-09-21.
- [7] exo contributors. exo: Run Frontier AI Locally, 2026. URL <https://github.com/exo-explore/exo>. Accessed 2026-09-21.
- [8] Zhiyuan Fang, Yuegui Huang, Zicong Hong, Yufeng Lyu, Wuhui Chen, Yue Yu, Fan Yu, and Zibin Zheng. Klotski: Efficient Mixture-of-Expert Inference via Expert-Aware Multi-Batch Pipeline, 2025. URL <https://arxiv.org/abs/2502.06888>. Accessed 2026-09-21.
- [9] Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Roziere, David Lopez-Paz, and Gabriel Synnaeve. Better and Faster Large Language Models via Multi-token Prediction, 2024. URL <https://arxiv.org/abs/2404.19737>. Accessed 2026-09-21.
- [10] Zongle Huang, Lei Zhu, Zongyuan Zhan, Ting Hu, Weikai Mao, Xianzhi Yu, Yongpan Liu, and Tianyu Zhang. MoESD: Unveil Speculative Decoding’s Potential for Accelerating Sparse MoE, 2025. URL <https://arxiv.org/abs/2505.19645>. Accessed 2026-09-21.
- [11] Intel Corporation. Intel Core Ultra X7 Processor 358H: Specifications, 2026. URL <https://www.intel.com/content/www/us/en/products/sku/245527/intel-core-ultra-x7-processor-358h-18m-cache-up-to-4-80-ghz/specifications.html>. Accessed 2026-09-21.
- [12] Intel Corporation. Industrial and Robotics Innovation with Intel Core Ultra Processors (Series 3), 2026. URL <https://builders.intel.com/docs/networkbuilders/industrial-and-robotics-innovation-with-intel-core-ultra-processors-series-3-1767869217.pdf>. Accessed 2026-09-21.
- [13] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast Inference from Transformers via Speculative Decoding, 2023. URL <https://proceedings.mlr.press/v202/leviathan23a.html>. Accessed 2026-09-21.
- [14] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty, 2024. URL <https://arxiv.org/abs/2401.15077>. Accessed 2026-09-21.

- [15] Zonghang Li, Wenjiao Feng, Mohsen Guizani, and Hongfang Yu. TPI-LLM: Serving 70B-scale LLMs Efficiently on Low-resource Edge Devices, 2024. URL <https://arxiv.org/abs/2410.00531>. Accessed 2026-09-21.
- [16] Zonghang Li, Tao Li, Wenjiao Feng, Mohsen Guizani, and Hongfang Yu. PRIMA.CPP: Speeding Up 70B-Scale LLM Inference on Low-Resource Everyday Home Clusters, 2025. URL <https://arxiv.org/abs/2504.08791v1>. Accessed 2026-09-21.
- [17] Bradley McDanel, Steven Li, Sruthikesh Surineni, and Harshit Khaitan. MoE-Spec: Expert Budgeting for Efficient Speculative Decoding, 2026. URL <https://arxiv.org/abs/2602.16052>. Accessed 2026-09-21.
- [18] Ivan Novikov. MLPMoE: Zero-Shot Architectural Metamorphosis of Dense LLM MLPs into Static Mixture-of-Experts, 2025. URL <https://arxiv.org/abs/2511.21089>. Accessed 2026-09-21.
- [19] OpenVINO contributors. OpenVINO 2026.3.1: Compressed Fused Three-GEMM MoE Primitive, 2026. URL https://github.com/openvinotoolkit/openvino/blob/2026.3.1/src/plugins/intel_gpu/include/intel_gpu/primitives/moe_3gemm_fused_compressed.hpp. Accessed 2026-09-22.
- [20] OpenVINO contributors. OpenVINO 2026.3.1: Tiled MoE Graph Lowering, 2026. URL https://github.com/openvinotoolkit/openvino/blob/2026.3.1/src/common/transformations/src/transformations/common_optimizations/convert_tiled_moe_block_to_gather_matmuls.cpp. Accessed 2026-09-22.
- [21] Matias Parij, Pawan Paudel, Tate Berenbaum, and Muthaiah Venkatachalam. Cascadia: A Control-Plane-Free Alternative to Hyperconverged AI Infrastructure, 2026. URL <https://github.com/labscommunity/cascadia-architecture-paper>. Accessed 2026-09-21.
- [22] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU, 2023. URL <https://arxiv.org/abs/2312.12456>. Accessed 2026-09-21.
- [23] Thinking Machines Lab. Inkling: Our Open-Weights Model, 2026. URL <https://thinkingmachines.ai/news/introducing-inkling/>. Accessed 2026-09-21.
- [24] Thinking Machines Lab. Inkling Model Card, 2026. URL <https://huggingface.co/thinkingmachines/Inkling>. Accessed 2026-09-21.
- [25] Wenxin Wang, Yule Hou, Yu Ji, Peng Qu, and Youhui Zhang. Achieving Cloud-Grade SLOs for Local Mixture-of-Experts Inference through CPU-GPU Hybrid Design, 2026. URL <https://arxiv.org/abs/2606.10493>. Accessed 2026-09-21.
- [26] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An Insightful Visual Performance Model for Floating-Point Programs and Multicore Architectures, 2009. URL <https://escholarship.org/uc/item/5tz795vq>. Accessed 2026-09-21.
- [27] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models, 2023. URL <https://proceedings.mlr.press/v202/xiao23c.html>. Accessed 2026-09-22.

- [28] Zheming Yang, Yunqing Hu, Sheng Sun, and Wen Ji. EC2MoE: Adaptive End-Cloud Pipeline Collaboration Enabling Scalable Mixture-of-Experts Inference, 2025. URL <https://arxiv.org/abs/2508.06024>. Accessed 2026-09-21.
- [29] Rongjie Yi, Liwei Guo, Shiyun Wei, Ao Zhou, Shangguang Wang, and Mengwei Xu. EdgeMoE: Empowering Sparse Large Language Models on Mobile Devices, 2025. URL <https://arxiv.org/abs/2308.14352v2>. Accessed 2026-09-21.
- [30] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A Distributed Serving System for Transformer-Based Generative Models, 2022. URL <https://www.usenix.org/conference/osdi22/presentation/yu>. Accessed 2026-09-21.
- [31] Zhengyan Zhang, Yankai Lin, Zhiyuan Liu, Peng Li, Maosong Sun, and Jie Zhou. MoEfication: Transformer Feed-forward Layers are Mixtures of Experts, 2022. URL <https://aclanthology.org/2022.findings-acl.71/>. Accessed 2026-09-21.
- [32] Ruidong Zhu et al. MegaScale-Infer: Serving Mixture-of-Experts at Scale with Disaggregated Expert Parallelism, 2025. URL <https://arxiv.org/abs/2504.02263>. Accessed 2026-09-21.